

Relatório técnico do protótipo

Arquitetura, dados B3/CVM/BCB, machine learning temporal, métodos quantitativos, QKP exato, PostgreSQL, segurança e execução local.

Arquitetura modular monolith
Backend FastAPI + Python 3.12
Persistência PostgreSQL
Otimização SCIP + branch-and-bound exato
Front-end Next.js + TypeScript
Execução Docker/Podman Compose

Escopo: software de pesquisa. Não constitui recomendação de investimento e não promete retorno futuro.

1. Escopo construído

O protótipo foi estruturado para ser funcional em uma máquina local e, ao mesmo tempo, não criar uma arquitetura descartável caso o projeto evolua para um produto comercial. A prioridade foi separar corretamente dados, pesquisa quantitativa, machine learning, otimização, persistência e apresentação.

O sistema tem quatro etapas centrais: reconstruir a informação que existia em determinada data; estimar a atratividade relativa de cada ação; selecionar de forma exata uma combinação de ativos considerando interações de risco; e calcular os pesos finais da carteira com um algoritmo específico de alocação.

Essa separação evita um erro comum em projetos de “IA para ações”: um único modelo recebe tudo, produz uma carteira e torna impossível descobrir de onde veio o resultado. Aqui cada camada pode ser comparada contra um baseline independente.

2. Arquitetura

Foi adotado um **modular monolith**. O sistema local sobe quatro componentes principais: PostgreSQL, Valkey, API e front-end. O Valkey é usado somente para controles efêmeros, como rate limiting; PostgreSQL permanece o system of record. A lógica quantitativa fica em um pacote independente (`portfolio_core`) e não conhece FastAPI ou Next.js.

Microserviços foram evitados porque ainda não existe necessidade operacional que justifique fila distribuída, descoberta de serviços, tracing entre dezenas de processos, versionamento de contratos e deploy independente. As fronteiras internas já existem, portanto módulos de ingestão, treino ou otimização podem ser extraídos para workers/serviços depois sem reescrever a lógica de domínio.

3. Linguagens e ferramentas

Python 3.12

Python é usado para dados, pesquisa, machine learning, risco e integração com SCIP. A versão 3.12 foi escolhida de forma conservadora: PySCIPOpt possui distribuição binária madura para essa versão em plataformas comuns, reduzindo a dificuldade de instalar o solver exato.

C++23

Há uma implementação exata independente de branch-and-bound em C++23. Ela funciona como referência verificável para instâncias pequenas e cria uma base para otimizações especializadas futuras. O backend principal continua sendo SCIP, pois resolve de forma muito mais robusta o modelo financeiro completo com cardinalidade, orçamento e limites adicionais.

TypeScript + Next.js

O front-end usa Next.js e TypeScript. A interface foi escrita sem um framework visual pesado para reduzir dependências e manter controle sobre ergonomia, responsividade e identidade visual. O resultado é um dashboard leve, com ranking, scores, portfolio builder, status do solver e composição da carteira.

4. PostgreSQL

PostgreSQL é o banco de dados da aplicação. Usuários, sessões, snapshots utilizados pela API, carteiras e posições são entidades relacionais e ficam no PostgreSQL.

O histórico bruto da B3 não foi colocado no banco transacional. Ele é preservado em ZIP e Parquet como arquivos analíticos imutáveis. Parquet não substitui o PostgreSQL como database da aplicação: ele funciona como formato de

armazenamento de datasets grandes utilizados pelo pipeline de pesquisa.

Essa escolha evita aumentar custo de ingestão e manutenção do banco apenas para armazenar milhões de linhas que são normalmente lidas em lote pelo treinamento.

5. Integração com dados históricos da B3

O projeto integra os arquivos oficiais COTAHIST anuais. A B3 documenta o arquivo com registros de 245 bytes e campos posicionais para data, BDI, ticker, mercado, empresa, especificação, preços, número de negócios, quantidade, volume, ISIN e outros dados.

O parser implementado segue essas posições oficiais. Ele mantém registros do mercado à vista e filtra especificações compatíveis com ações/units (ON, PN, variações de PN e UNT).

A ingestão foi desenhada com rastreabilidade:

- ZIP original permanece imutável;
- SHA-256 e URL de origem são salvos em manifest;
- dados parseados ficam em camada `silver`;
- features e resultados ficam em `gold`;
- uma derivação nunca sobrescreve o arquivo fonte.

A própria B3 informa que as cotações históricas não são ajustadas por inflação ou proventos como dividendos, bonificações e direitos. Por isso o protótipo consegue utilizar dados reais da B3, mas **não deve ser usado para afirmar performance total-return em produção** até existir uma camada de eventos corporativos com direito de uso compatível com o produto.

6. CVM e fundamentos

Foram adicionados clientes para ITR e DFP da CVM. O ponto mais importante é preservar `DT_RECEB`.

Um demonstrativo referente a 31 de março não pode aparecer como feature em 1º de abril se só foi recebido pela CVM semanas depois. O contrato de dados do projeto exige:

```
DT_RECEB <= prediction_date
```

A ligação entre ticker e cadastro CVM não é feita por fuzzy matching silencioso. Nomes de empresas mudam, holdings são reorganizadas e tickers trocam. Um matching automático errado é mais perigoso que uma feature ausente porque cria números plausíveis associados ao emissor incorreto. Para uma versão comercial, esse bridge deve ser uma tabela governada, com vigência temporal e aprovação.

7. Banco Central

O módulo BCB utiliza o serviço SGS e já possui configuração inicial para SELIC, IPCA e dólar. A camada macro fica separada para que, no futuro, também seja possível controlar o momento de divulgação de cada série, frequência e revisões.

8. Machine learning

O baseline principal não é uma LSTM ou Transformer. Foi escolhido um ensemble de LightGBM, CatBoost e Ridge.

Isso é proposital. O dataset que combina ações, features técnicas, fundamentos e macro é essencialmente tabular e ruidoso. Boosted trees são referências fortes para esse cenário e são muito mais fáceis de validar, interpretar e treinar com volume moderado de dados. Ridge fornece um componente simples de baixa variância.

O target é retorno futuro excedente, e não preço exato futuro. No protótipo autocontido, o retorno médio cross-sectional funciona como proxy de mercado. Uma versão de pesquisa final deve usar um benchmark explicitamente definido (por exemplo, IBOV ou CDI, conforme o mandato).

Features implementadas

- retornos de 5, 21, 63, 126 e 252 pregões;
- momentum 12-1;
- volatilidade móvel;
- distância para médias móveis;
- proxies de volume e número de negócios;
- rankings cross-sectional das principais variáveis.

O módulo fundamental implementa um conjunto conservador de ratios baseados em contas padronizadas (margem líquida/bruta, proxies de ROE/ROA, asset turnover e escalas de receita/ativos/patrimônio). Eles são incorporados automaticamente ao treino quando existe um ticker/CVM bridge revisado e temporalmente válido. Uma versão comercial deve ampliar essa taxonomia por setor e manter governança sobre as regras contábeis.

9. Validação temporal

Não existe `train_test_split(shuffle=True)` no fluxo de pesquisa.

O treinamento e o backtest usam janelas temporais. Rank IC, top-decile e top-minus-bottom são calculados dentro de cada data de previsão e só depois agregados no tempo, evitando ranquear uma ação de janeiro contra outra de novembro.

O projeto inclui tuning opcional com Optuna. O tuning usa janelas expansivas anteriores e mantém o ano final completamente intocado como holdout; o script de treino só aceita o arquivo de hiperparâmetros quando o holdout registrado coincide com o ano de validação final. O pré-processamento aprende mediana/escala apenas no treino. O backtester walk-forward treina usando anos anteriores ao ano que será previsto.

Os critérios de promoção devem ser principalmente:

- Rank IC/Spearman;
- retorno do top decile;
- top-minus-bottom;
- estabilidade por período/setor;
- turnover;
- retorno líquido de custos;
- Sharpe, Sortino e drawdown da carteira.

Um R^2 alto isoladamente não é considerado evidência suficiente.

10. Algoritmos quantitativos

O core contém componentes para comparar ML contra abordagens quantitativas conhecidas:

- momentum de médio prazo;
- preferência por menor volatilidade como sinal defensivo;
- covariância Ledoit-Wolf para reduzir instabilidade amostral;
- Hierarchical Risk Parity (HRP);
- inverse volatility;
- minimum variance;
- Black-Litterman para converter previsões do ML em views com incerteza.

O sistema deve sempre comparar estratégias complexas contra equal weight e fatores simples. Caso ML + QKP não supere alternativas simples fora da amostra e depois de custos, a arquitetura permite provar isso em vez de mascarar o resultado.

11. QKP exato

A seleção resolve um problema binário quadrático:

$$\max \sum(p_{i,x_i}) + \sum(q_{ij}x_i x_j)$$

p_i combina retorno excedente previsto, penalidade de incerteza e uma contribuição pequena e explícita do score quantitativo (momentum + baixa volatilidade). Antes do solver, o candidate universe também combina score ML, quant e liquidez. Isso permite testar ML e fatores clássicos conjuntamente sem esconder cada componente.

q_{ij} representa o efeito de combinar dois ativos. No protótipo, correlação positiva gera penalidade e correlação negativa pode fornecer benefício de diversificação. Assim, o solver não escolhe somente as ações mais bem ranqueadas individualmente.

SCIP

SCIP é o backend principal. Os produtos binários são linearizados exatamente com uma variável y_{ij} e três restrições:

$$y_{ij} \leq x_i$$

$$y_{ij} \leq x_j$$

$$y_{ij} \geq x_i + x_j - 1$$

Com isso o modelo aceita coeficientes quadráticos positivos e negativos, orçamento, cardinalidade e limites por setor. A API só marca `exact=true` quando o status retornado comprova otimalidade.

Solver independente

Também existem implementações exatas de branch-and-bound em Python e C++²³. O upper bound é propositalmente conservador: ele soma todas as contribuições positivas ainda possíveis ignorando algumas restrições. Isso pode ser frouxo, mas nunca subestima o melhor futuro possível; portanto, a poda continua exata.

A implementação Python foi comparada contra brute force em 20 instâncias aleatórias de 8 itens, incluindo interações negativas, sem divergência de objetivo. O solver C++ foi compilado e seu teste unitário passou durante a construção deste artefato.

12. Separar seleção de sizing

O QKP decide quais ações entram. Depois um segundo algoritmo calcula os pesos.

A alocação padrão é HRP. Também existem inverse volatility e minimum variance. Essa decisão facilita experimentos controlados, por exemplo:

- ML + equal weight;
- ML + QKP + equal weight;
- ML + QKP + HRP;
- momentum + QKP + HRP.

Se a seleção QKP não melhorar o resultado, isso aparece claramente.

13. Backtester

O backtester mensal recebe sinais datados, aplica-os somente depois da data do sinal, mede turnover e cobra custo de transação em basis points.

O script de ML realiza retraining walk-forward. Antes de uma versão comercial, ainda devem ser adicionados:

- preços total-return ajustados;
- histórico de delistings e mudanças de ticker;
- universo investível reconstruído em cada data;
- bid/ask e slippage dependente de liquidez;
- benchmark com timestamps idênticos;
- regras tributárias caso sejam relevantes ao produto oferecido.

14. Segurança

Mesmo sendo local, a base já incorpora práticas que evitam retrabalho:

- Argon2id para senhas;
- token de sessão aleatório;
- somente hash do token fica no PostgreSQL;
- cookie de sessão HttpOnly;
- SameSite;
- CSRF por double-submit + token de sessão nas operações autenticadas;
- CORS por allowlist;
- security headers;
- Secure e HSTS fora do ambiente de desenvolvimento;
- segredos via variáveis de ambiente;
- containers da API e front-end sem root;
- rate limiting distribuído via Valkey para autenticação e otimização;
- optimizer protegido por sessão + CSRF quando DEMO_MODE=false.

Para Internet pública ainda faltam persistência completa de auditoria, TLS/reverse proxy, gestão central de secrets, rotina formal de backup/restore, SAST contínuo, revisão LGPD e teste externo de segurança.

15. Front-end

A interface foi construída para parecer uma ferramenta de pesquisa e não um terminal de trading. A tela principal mostra:

- estado dos motores ML, optimizer, risco e dados;
- ranking das ações;
- score do modelo, alpha esperado, volatilidade e setor;
- controles de capital, número de ações e aversão ao risco;
- status do solver exato;
- composição e pesos da carteira.

A informação “ótimo provado” fica visível. Isso é importante porque uma solução heurística e uma solução ótima não devem ser apresentadas como se fossem equivalentes.

16. Escalabilidade

O desenho atual suporta evolução sem reescrever o core. Um caminho natural seria:

1. mover raw/silver/gold para object storage;
2. executar ingestão, treino e backtests em workers;
3. criar scheduler/orchestrator;
4. manter PostgreSQL como system of record e escalar com particionamento/replicas quando necessário;
5. separar o optimizer em worker caso exista fila de solves;
6. adicionar registry de modelos e artefatos;
7. adicionar CDN/edge no front-end.

Não há motivo para pagar esse custo operacional no primeiro protótipo.

17. Pontos obrigatórios antes de comercializar

A base técnica pode evoluir para produto, mas existem gates independentes de engenharia:

- confirmar licença/termos da B3 para uso comercial, derivação e eventual redistribuição;
- validar com profissional jurídico o enquadramento perante CVM;
- completar corporate actions/total return e survivorship-safe universe;
- demonstrar resultados repetidos fora da amostra e após custos;
- implementar governança e auditoria de modelos;
- concluir controles LGPD, backup, incident response e segurança externa.

18. O que foi validado neste artefato

- o código Python foi submetido a `compileall` para validação sintática;

- foi adicionada suíte de testes para parser B3, métricas cross-sectional e QKP; a execução completa dessa suíte requer as dependências Python declaradas no container;
- o solver C++23 foi compilado via CMake;
- o teste unitário do solver C++ passou;
- o branch-and-bound Python foi comparado contra brute force em 20 instâncias aleatórias pequenas e produziu o mesmo ótimo;
- as posições do parser COTAHIST foram conferidas contra o layout oficial da B3.

O ambiente usado para montar o artefato não possui acesso externo para instalar pacotes npm/pip durante a validação final. Por isso a compilação completa do Next.js e a instalação do Compose precisam ocorrer na máquina local do usuário. Os Dockerfiles já contêm esse processo.

19. Fontes técnicas principais

- B3 - Cotações históricas: https://www.b3.com.br/pt_br/market-data-e-indices/servicos-de-dados/market-data/historico/mercado-a-vista/cotacoes-historicas/
- Layout COTAHIST: https://www.b3.com.br/data/files/33/67/B9/50/D84057102C784E47AC094EA8/SeriesHistoricas_Layout.pdf
- CVM Dados Abertos: <https://dados.cvm.gov.br/>
- BCB SGS: <https://api.bcb.gov.br/dados/serie/bcdata.sgs.{codigo}/dados>
- PySCIPOpt: <https://pyscipopt.readthedocs.io/en/stable/install.html>
- SCIP: <https://www.scipopt.org/>
- Gu, Kelly e Xiu - Empirical Asset Pricing via Machine Learning.
- Jegadeesh e Titman - Returns to Buying Winners and Selling Losers.
- Ledoit e Wolf - trabalhos de shrinkage de matriz de covariância.
- Lopez de Prado - Building Diversified Portfolios that Outperform Out of Sample (HRP).
- Black e Litterman - framework de otimização global de portfólio.